Members: Sofia Wolfel (smw318), Vivian Zhao (vz57)
Professor Cowan
Intro to AI
13 October 2025

Project 1 Writeup

Part 1: Optimality Strategy Design Choices

Work mainly contributed by Sofia Wolfel

For the optimal solution, we implemented a belief system using heuristics to improve upon our baseline A* method. Since the bot doesn't know its current location, we have to maintain a set of possible positions it could be in, or a belief state, rather than track a singular known position. By minimizing sets of belief states, the bot can rule out sets of possible locations that have higher uncertainty and more efficiently explore the states with the smaller sets of belief states, which ensures a shorter path from the bot's current possible location to the randomly chosen dead-end goal.

Our optimality strategy uses informed search through heuristic functions designed to prioritize belief states that are smaller in size and less spatially dispersed. We calculated the spatial spread of the belief state by computing max(r+c) - min(r+c) and max(r-c) - min(r-c). This calculation shows the maximal diagonal extent of the belief state, which measures how spread out the bot's possible locations are. Bigger spreads usually have higher uncertainty as it's easier to figure out where the bot is with a smaller spread. However, a smaller spread doesn't always guarantee a shorter path to localization if there is still a big number of possible positions. This is where heuristic comes into play; we added a weighted penalty based on the number of positions left, encouraging the bot to make moves that shrink the belief set faster. In the code, it is implemented as max_dist + 3 * len(L), where max_dist measures spatial spread, len(L) counts the number of possible positions, and the weight of 3 balances the two components. The result of this combination would effectively help the bot localize itself in the fewest possible moves. This heuristic is admissible, as it never overestimates the number of moves required to fully localize the bot. The first piece of evidence of admissibility is that the spatial spread provides a lower-bound estimate of the minimum number of steps required to reduce all positions into a single location. And the second piece of evidence is that the belief size counts only remaining possibilities, as reducing this set cannot be completed in fewer moves than considered by search. By always choosing to expand belief state with the lowest f = g + h (f is estimated total cost to goal while following current path, g is number of moves completed so far, and h is heuristics), the algorithm explores the best paths first while ensuring that no potentially shorter sequences of actions is ignored.

We also update our belief sets based on the bot's action at each step. For example, if the bot hits a wall and remains in place, the belief set will update accordingly. We represent belief states as frozensets to allow efficient storage and comparison in the priority queue used by A*, and the search continues until the belief set contains only one position, or full localization. Using A* informed search also ensures that we don't waste moves exploring states that don't help improve localization. Lastly, we use pruning and

tie-breaking to keep track of the minimal g value for each belief state and avoid re-expanding explored states.

Part 2: Efficiency Strategy Design Choices

Work mainly contributed by Vivian Zhao

For the efficiency strategy, we implemented a greedy algorithm that shrinks the belief set directly by choosing an action at each step that results in the fewest remaining possible locations for the bot. In other words, we make locally optimal decisions at each step until the belief set reaches a single point.

The bot's uncertainty is represented as a belief set L, which begins by consisting of all open cells in the grid. Additionally, we set up small lookup tables that record three things: a cell X the bot could currently be in, an action a, and the resulting cell Y after applying a to X. If Y is an open cell, the bot moves there; otherwise, it stays at its original cell X. Using these lookup tables allows the entire belief set to be updated with simple array lookups, which are fast and make the computation efficient.

For each hypothetical action a that the bot could attempt in $a \in \{U, D, L, R\}$, the algorithm checks the resulting set of possible spots and picks the action that would shrink the set the most. This greedy approach pushes for successful localizations much quicker than the baseline and optimality strategies, as each action evaluation becomes a linear pass over the existing belief indices. Although this method sacrifices look-ahead for actions that can lead to bigger reductions in $|L|$ later on, it scales better on larger ships and avoids long-term decisions that would become too expensive in the long-run and potentially slow down the process exponentially (worst case).

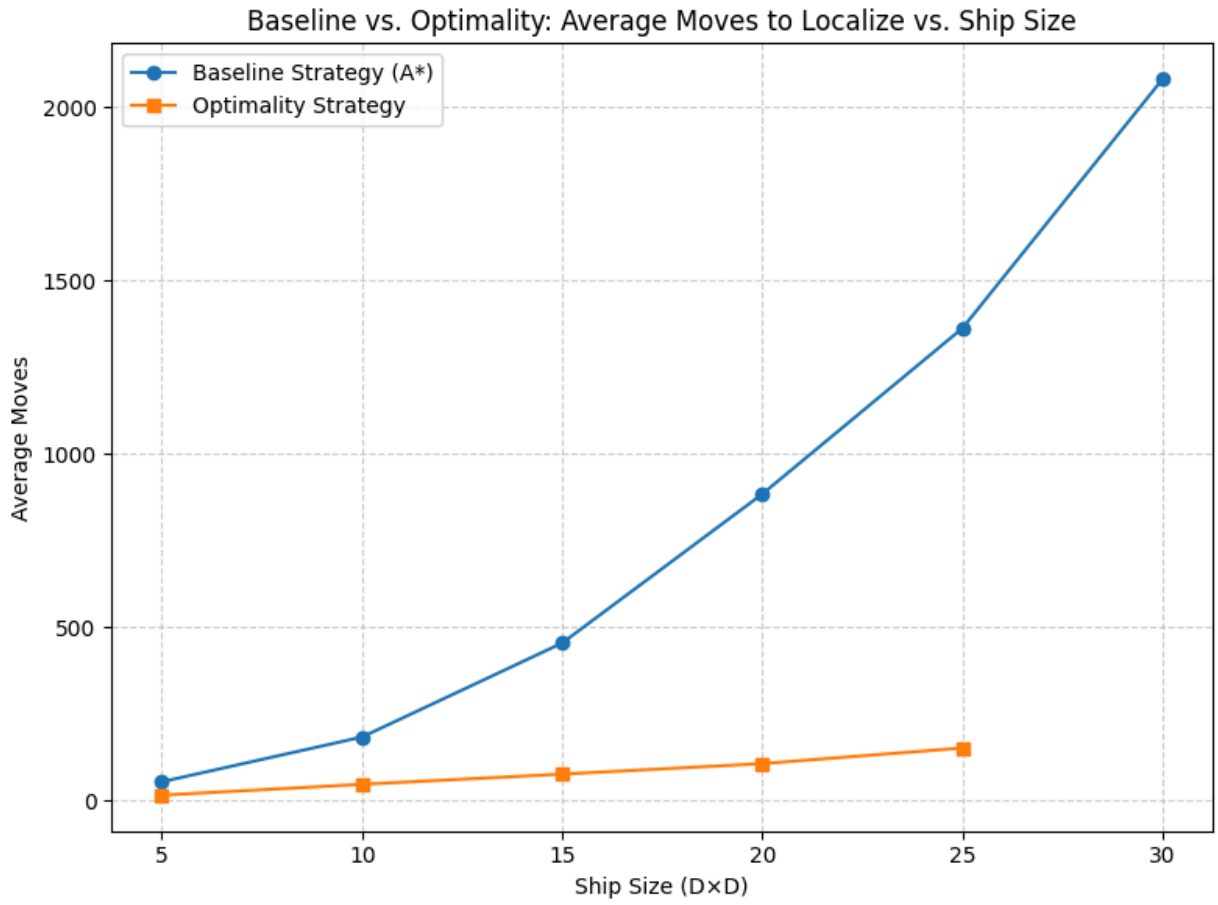
Since greedy methods can run the risk of becoming stuck in loops over similar belief sets, we added a small set of *seen* beliefs. It helps perform a simple check: if choosing the best action means that the same one keeps repeating, then we pick a different action that yields an equally small belief set. This adds a small overhead while helping to prevent loops. However, we quickly realized that this does not fully prevent loops from occurring, so we also included a *max_moves* cutoff to halt the algorithm if it still becomes stuck. As one more safeguard, we introduced *escape_every = k*: for every k steps, if $|L|$ hasn't decreased since the last check, we force an escape move by choosing one of the actions that keeps $|L'|$ minimal (or near minimal within +1 if necessary), and leads to a new belief that is not in the *seen* set.

To further simplify the logic, we mapped each open cell to an integer index rather than keeping them as separate (row, col) pairs. Managing integers and array indexes is faster and easier for memory than hashing many (row, col) tuples, which can make a significant difference as grid size increases. Accounting for larger sized grids is also why this algorithm is based on belief states rather than pathfinding between two known points on the grid. Finding direct paths between two given points, such as repeated A*, would involve additional computation costs which can be avoided by using belief states to focus on quicker uncertainty reduction.

Part 3: Baseline vs. Optimality Strategy Performance Comparison

Work mainly contributed by Sofia Wolfel

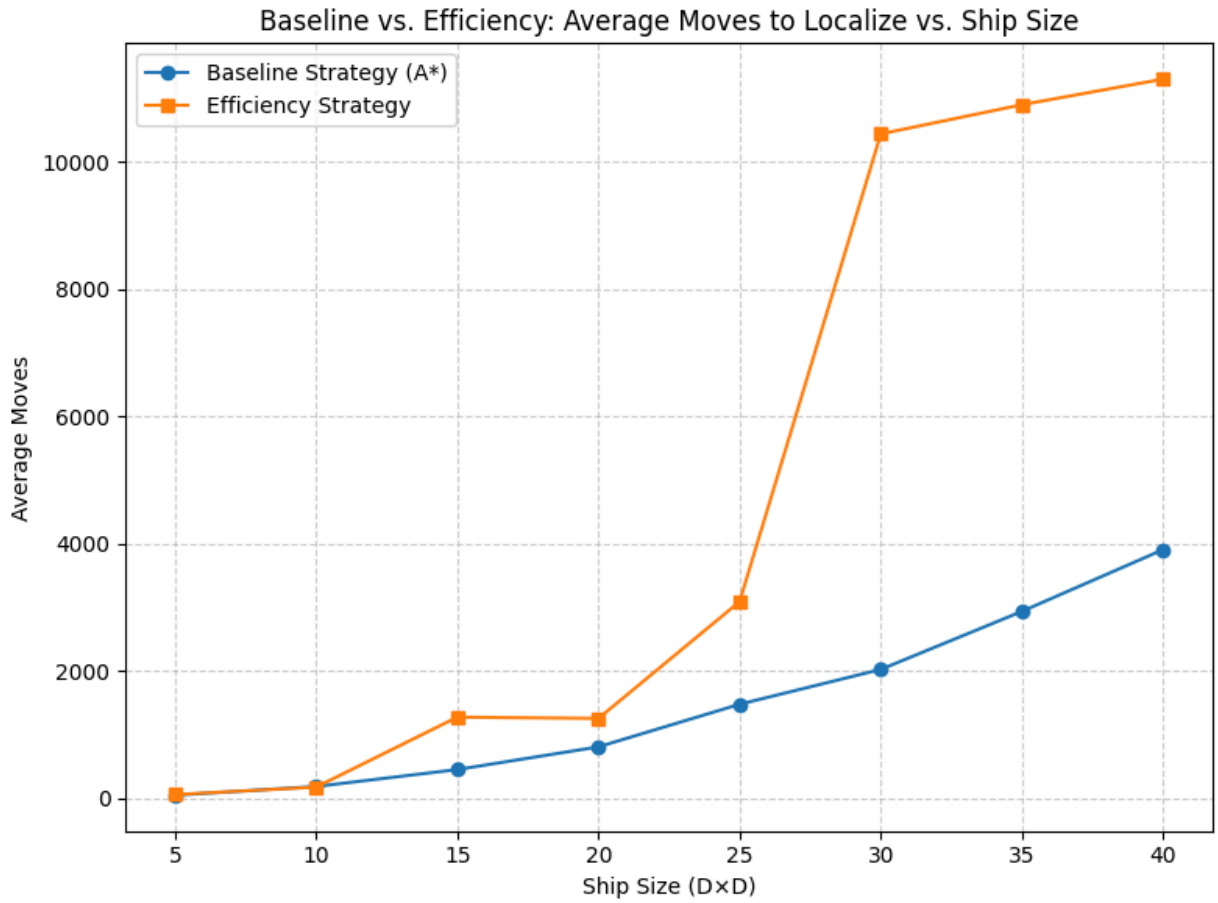
Comparing the average moves required to localize the bot between the optimal and the baseline method, the optimal method consistently outperforms the baseline method, even though the optimal method does not work on ship sizes bigger than a 25 x 25 grid. Based on estimation looking at the graph, the optimal method took around 200 moves to localize the bot while the baseline method took around 1750 moves to localize the bot for a ship sized at 25, which makes the optimal almost 9 times faster than the baseline at that ship size. For smaller grids (e.g., D = 5), the methods seem to perform similarly, but it seems that as the ship size increases, the baseline method grows exponentially while the optimal method grows almost linearly. This difference is likely due to the baseline method using uninformed search which leads to the bot exploring a large amount of redundant or uncertain states while the optimal method uses heuristics to prioritize states with lower uncertainty as well as to minimize redundant exploration.



Part 4: Baseline vs. Efficiency Strategy Performance Comparison

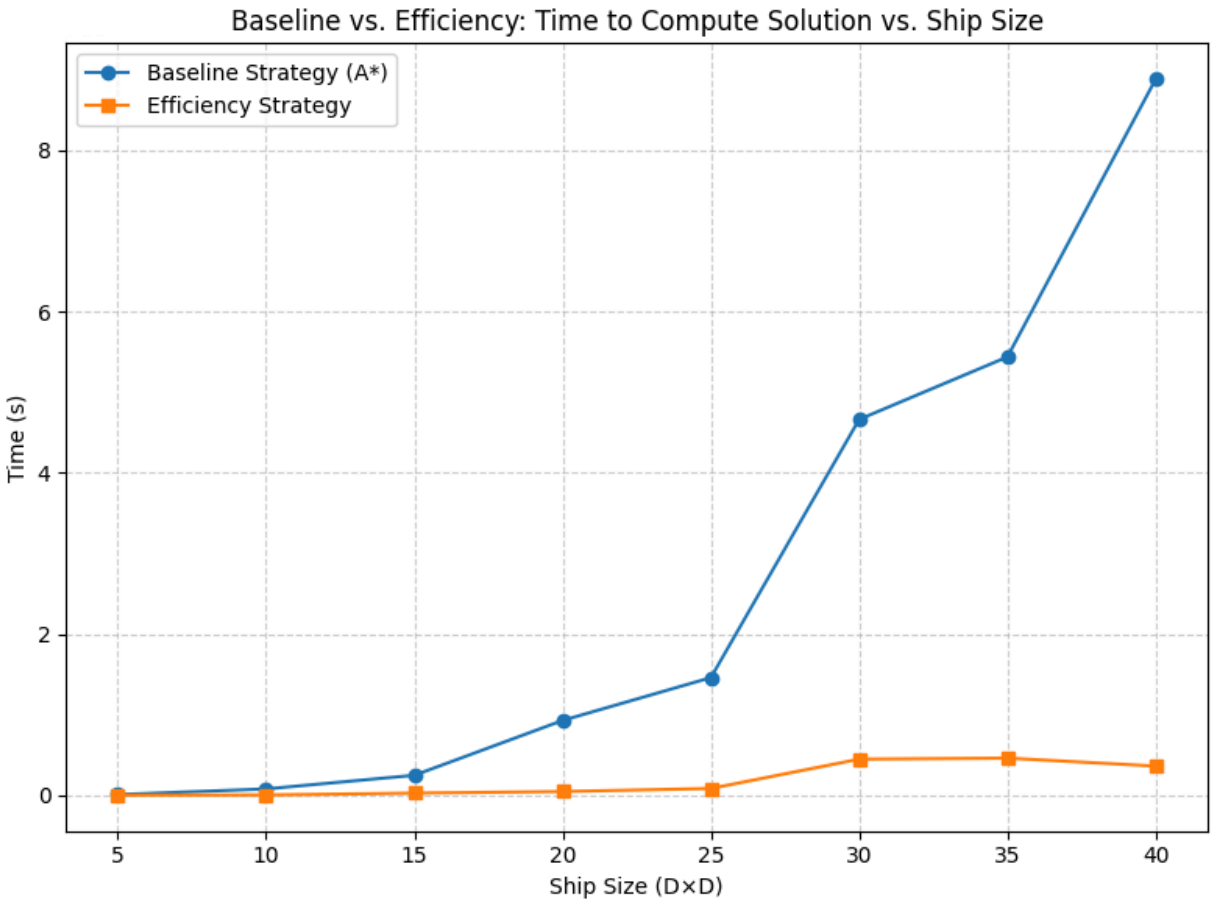
Work mainly contributed by Vivian Zhao

After implementing and testing the baseline and efficiency strategies across various ship sizes, it's clear that the baseline strategy outperforms the efficiency strategy in terms of average moves needed to localize the bot. Therefore, although the efficiency strategy may be faster in computing the solution, it still requires many more moves to reach that solution due to its greedy approach.



According to the graph above, the two strategies seem to perform similarly on smaller ships sizes, where $n = 5$ through 20. Both result in total moves of less than 2000. However, the gap between the two strategies widens dramatically as the ship size increases. At a ship size of 25, the baseline strategy requires approximately 1500 moves, while the efficiency strategy requires around 3000 moves—roughly double that amount. This gap only continues to grow significantly, and by a ship size of 40, the difference in average moves reaches more than 7000 moves, as the baseline requires around 4000 moves and the efficiency strategy requires around 11500 moves.

Though the efficiency strategy requires more moves to localize the bot than the baseline strategy, it manages to achieve this with much faster computation times across all of the tested ship sizes. The graph below shows the main advantage of this efficiency strategy: computational speed over solution optimality.



Based on the graph, the two strategies showed relatively similar results in computation time for smaller ship sizes (5-25), which was also the case for the previous graph. But once the ship size increases to 30 and above, the computation time for the baseline strategy shoots up almost exponentially, while the efficiency strategy maintains a somewhat consistent computation time for all the tested ship sizes. By the end, with a ship size of 40, the baseline strategy requires approximately 9 seconds while the efficiency strategy requires less than 1 second.

Part 5: Additional Algorithms

Work contributed by Sofia Wolfel and Vivian Zhao

One way to find the smallest set of locations that forces the optimal solution to take as many moves as possible before localizing the bot is to select sets of possible locations that maximize heuristic uncertainty.

We implemented an algorithm that selects pairs of dead-ends that are farthest apart from each other as worst-case starting positions. Since dead-ends have only one open neighbor, they are forced to go one direction as the location has a lot of constraints. Therefore, the belief should shrink slowly, leading to the bot taking more moves to localize. However, we don't think the algorithm ended up taking more steps compared to optimal, which we had gotten stuck on. However, after running the results through ship sizes 10, 15, 20, 25, and 30, we discovered that the worst-case cells tend to be corner-to-corner or near-opposite edges and may have to do with symmetry in the grid. Here are the results of testing various ship sizes on the same seed:

10×10	{(5, 5), (3, 9)}
15×15	{(9, 12), (0, 0)}
20×20	{(0, 3), (17, 14)}
25×25	{(3, 24), (24, 1)}
30×30	{(0, 29), (29, 2)}

To find the smallest set of locations that forces the efficiency strategy to run for as long as possible before localizing the bot, we implemented a simplified search algorithm that focuses on finding pairs of starting locations. The algorithm first generates all possible pairs of open cells, but if the grid is larger, then it samples random pairs. For each pair, we ran a variant of our original efficiency greedy algorithm that would return the total number of moves that are required to localize the bot. The pair that needs the most moves is then recorded as the 'hardest' subset. By definition, this approach is minimal since it only searches for pairs, and pairs cannot be reduced further without eliminating the initial ambiguity. For smaller grids that have fewer open cells, the algorithm iteratively tests all pairs. For larger grids, random sampling allows the search to remain practical in computation without sacrificing the ability to find these 'hard' pairs.

```
Size 5: pair = (2, 2), (4, 2) | moves = 72
Size 10: pair = (3, 0), (9, 8) | moves = 220
Size 15: pair = (5, 12), (10, 1) | moves = 1077
Size 20: pair = (17, 11), (18, 19) | moves = 1914
Size 25: pair = (10, 16), (9, 18) | moves = 1591
Size 30: pair = (7, 27), (1, 8) | moves = 3308
Size 35: pair = (10, 33), (1, 13) | moves = 21441
Size 40: pair = (39, 21), (6, 38) | moves = 13175
```

The output above shows the solutions we found for eight different ship sizes. The number of moves required for each worst-case scenario generally increases with ship size, though certain outliers like size 20 requiring more moves than size 25 highlight that grid complexity matters more than the size alone. A majority of the pairs also show some spatial separation—they are typically farther apart on the grid. This may explain why these pairs are ‘hard.’ However, distance does not seem like a reliable predictor, as there are other pairs located closer together that still require significant moves to localize.

What makes this localization problem hard in the efficiency sense is the presence of loops and repetition in structure in the grid. When multiple belief states look the same under certain actions, the greedy algorithm loses the ability to distinguish between these beliefs and may become stuck in a loop without making further progress towards localization.